FORMACIÓN INTERNA DEL EQUIPO

Manual de OpenCode V2 y OpenSpec

Toda la web en un solo documento, en el mismo orden que el menú lateral. La versión en línea siempre es la más reciente.

Edición de 8 de noviembre de 2026

> Para el método completo de OpenSpec, consulta la **Guía completa de OpenSpec**: **openspec-ochre.vercel.app**.

Diseño y desarrollo: webdespega.com

Documento generado desde la web

CONTENIDO

Índice

INICIO

01 OpenCode - Asistente de IA de código abierto

Guía en español de OpenCode V2, la herramienta de programación asistida por IA para la terminal

02 Ejemplos prácticos

Peticiones de ejemplo para tareas habituales con OpenCode V2

RUTA DE APRENDIZAJE

03 Ruta de aprendizaje

Un plan de cuatro semanas para dominar OpenCode V2

GUÍAS

04 Agentes

Agentes integrados de OpenCode V2 y cómo crear los tuyos

05 Configuración

Ficheros opencode.json, ámbitos, permisos y substituciones en OpenCode V2

06 Inicio rápido

Instala OpenCode V2, conecta un proveedor y ejecuta tu primera sesión

07 Skills

Instrucciones reutilizables en formato SKILL.md para OpenCode V2

08 TUI

Atajos de teclado, comandos de barra y gestión de sesiones en la interfaz de terminal de OpenCode V2

09 Zen y OpenCode Console

Cómo acceder a modelos de IA mediante OpenCode Console, con pago por uso

AVANZADO

10 Referencia de comandos

Comandos de la CLI de OpenCode V2, comandos de barra y comandos personalizados

11 FAQ y solución de problemas

Respuestas a preguntas frecuentes y diagnóstico de problemas en OpenCode V2

12 Servidores MCP

Conecta OpenCode V2 con servidores MCP locales o remotos

13 Migrar de V1 a V2

Pasos y equivalencias para pasar de opencode-ai 1.x a @opencode/cli

14 Flujos de trabajo

Un flujo recomendado para planificar, implementar y revisar cambios con OpenCode

OPENCODE + OPENSPEC

15 OpenCode + OpenSpec

Cómo combinar OpenCode con OpenSpec para planificar cambios antes de programarlos

16 Comandos del CLI

Comandos de la terminal de OpenSpec para listar, validar y archivar cambios

17 Flujo de trabajo

Explorar, proponer, implementar y archivar un cambio con los comandos /opsx-* de OpenCode

18 Instalación y configuración

Instala OpenSpec y prepara un proyecto para usarlo desde OpenCode

RECURSOS

19 Recursos

Documentación oficial y guías de esta web para aprender OpenCode

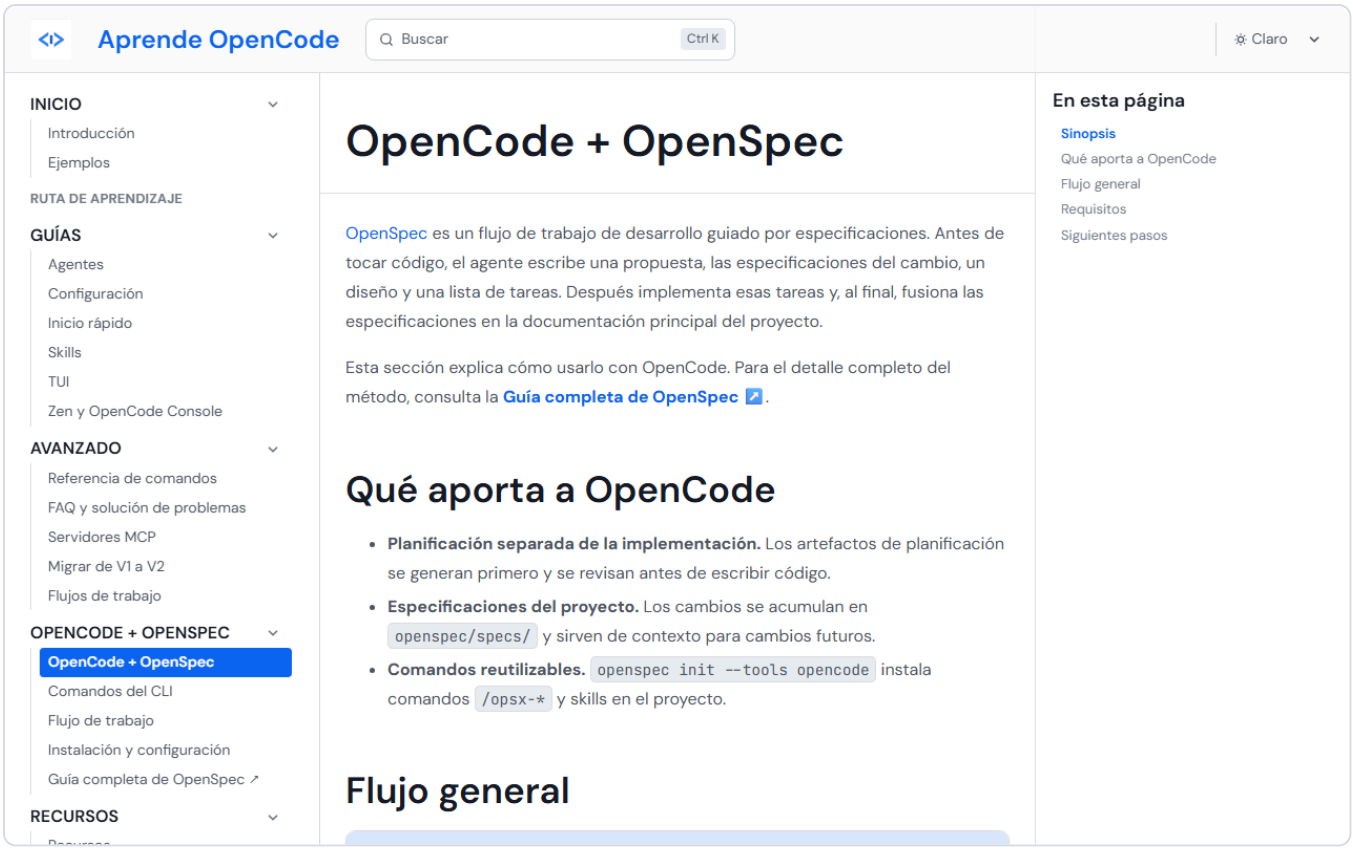
VISTA PREVIA

Así es la web

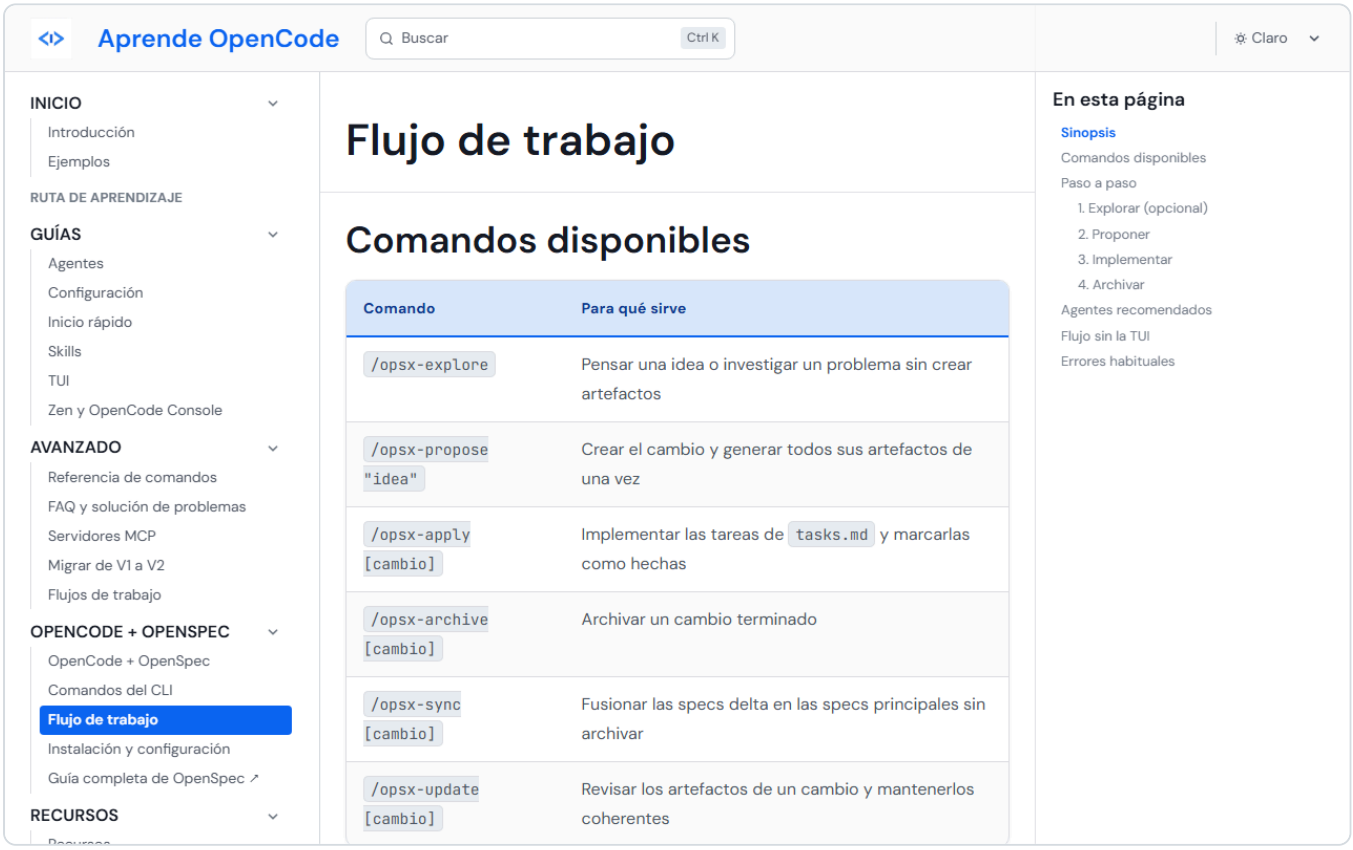
Capturas de la propia web, en escritorio y tema claro: la página de inicio, el menú lateral, el flujo de OpenSpec, una tabla de migración y el pie de página.



Página de inicio: punto de partida, con la introducción y los accesos principales.



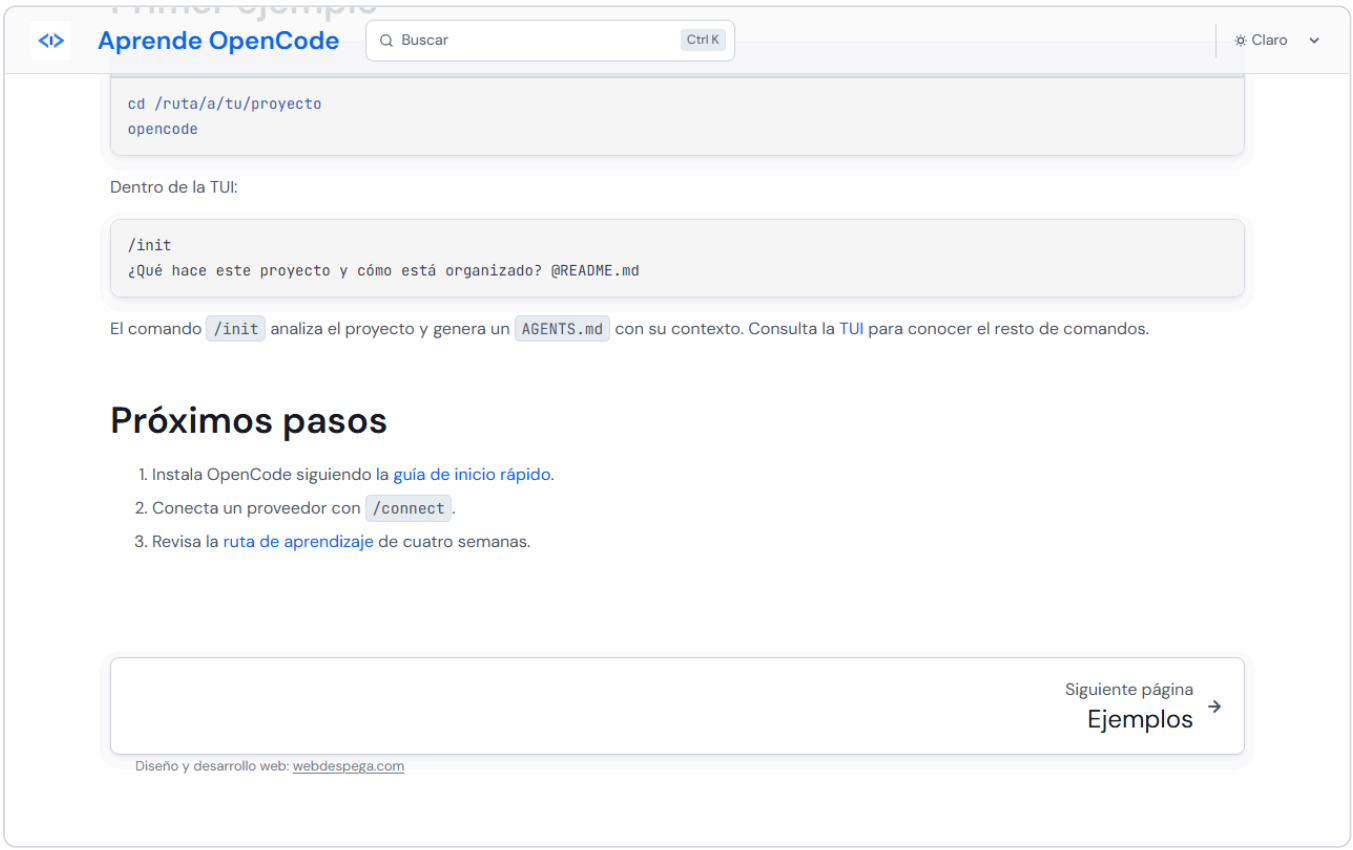
Menú lateral con el grupo «OpenCode + OpenSpec» desplegado.



Flujo de trabajo de OpenSpec: explorar, proponer, implementar y archivar un cambio.

Método V1	Equivalente V2
`npm install -g opencode-ai`	`npm install -g @opencode/cli`
`curl -fsSL https://opencode.ai/install | bash`	`curl -fsSL https://opencode.ai/v2/install | bash`
`brew install anomalyco/tap/opencode`	`brew install anomalyco/tap/opencode-v2`
`bun install -g opencode-ai`	`bun install -g --trust @opencode/cli`

Tabla de equivalencias de la guía de migración de V1 a V2.



Pie de página con el crédito de diseño y desarrollo.

CAPÍTULO 01 · INICIO

OpenCode – Asistente de IA de código abierto

Guía en español de OpenCode V2, la herramienta de programación asistida por IA para la terminal

---

Manual completo para leer o imprimir sin conexión: descarga el PDF.

Qué es OpenCode

OpenCode es una herramienta de programación asistida por IA que funciona en la terminal. Puedes elegir el modelo que quieras a través de tu proveedor o de OpenCode Console, y adaptar el comportamiento del agente con agentes, comandos, skills y servidores MCP.

Esta guía cubre **OpenCode V2** (paquete `@opencode/cli`). Si todavía usas la versión 1 (`opencode-ai`), empieza por Migrar de V1 a V2.

Mapa de la guía

Sección	Contenido
Inicio rápido	Instalación, primer arranque y actualización
TUI	Atajos, comandos de barra y sesiones
Configuración	`opencode.json`, permisos y ámbitos
Agentes	Agentes integrados y personalizados
Skills	Instrucciones reutilizables
Zen / Console	Acceso a modelos mediante OpenCode Console
Comandos	CLI y comandos personalizados
MCP	Servidores de herramientas externos
OpenCode + OpenSpec	Planificar cambios con especificaciones antes de programar

Nueva sección: OpenCode + OpenSpec

Si trabajas con especificaciones, la sección OpenCode + OpenSpec explica cómo usar los comandos `/opsx-*` para proponer, implementar y archivar cambios. Para el método completo, consulta la Guía completa de OpenSpec ↗.

Primer ejemplo

```
cd /ruta/a/tu/proyecto
opencode
```

Dentro de la TUI:

```
/init
¿Qué hace este proyecto y cómo está organizado? @README.md
```

El comando `/init` analiza el proyecto y genera un `AGENTS.md` con su contexto. Consulta la TUI para conocer el resto de comandos.

Próximos pasos

1. Instala OpenCode siguiendo la guía de inicio rápido.
2. Conecta un proveedor con `/connect`.
3. Revisa la ruta de aprendizaje de cuatro semanas.

Ejemplos prácticos

Peticiones de ejemplo para tareas habituales con OpenCode V2

Estos ejemplos son peticiones que puedes escribir en la TUI. Adáptalas al lenguaje y a la estructura de tu proyecto. Las referencias con `@` apuntan a ficheros o carpetas del proyecto.

Entender un proyecto

```
/init
Describe la arquitectura de este proyecto: capas, módulos y puntos de
entrada. @README.md
```

Depurar un error

```
Este error aparece al ejecutar los tests. Localiza la causa en @src/ y
propón una corrección sin editar nada todavía:
<pega aquí el mensaje de error>
```

Con el agente `plan`, la petición se queda en análisis. Cuando la propuesta sea correcta, pide la implementación con `build`.

Escribir tests

```
Escribe tests unitarios para @src/utils/validation.ts cubriendo casos
válidos, límites y errores.
```

Revisar cambios

```
Revisa los cambios sin confirmar del repositorio buscando errores de
lógica y de seguridad. Ordena los hallazgos por severidad.
```

Para repetir esta revisión, crea un comando personalizado. Consulta Comandos.

Refactorizar con plan

```
Propón cómo reducir la complejidad de @src/checkout/ sin cambiar su
comportamiento. Lista los pasos en orden.
```

Ejecutar sin abrir la TUI

```
opencode run --agent plan "Lista los endpoints HTTP definidos en este
proyecto"
```

Siguientes pasos

- Flujos de trabajo para organizar tareas más grandes.
- OpenCode + OpenSpec para cambios que necesitan especificaciones.

CAPÍTULO 03 · RUTA DE APRENDIZAJE

Ruta de aprendizaje

Un plan de cuatro semanas para dominar OpenCode V2

Ruta de aprendizaje de OpenCode

Cada semana tiene objetivos, tareas y una comprobación final. Avanza al ritmo que necesites.

Semana 1: Fundamentos

Objetivos

- Instalar OpenCode V2 y abrir la TUI en un proyecto.
- Conectar un proveedor.

Tareas

- ☐ Completar la guía de inicio rápido.
- ☐ Ejecutar `/init` en un proyecto y revisar el `AGENTS.md` generado.
- ☐ Practicar referencias con `@` y la tecla líder `ctrl+x` (ver TUI).

Comprobación: puedes hacer una pregunta sobre un fichero concreto y obtener una respuesta útil.

Semana 2: Flujo de trabajo

Objetivos

- Entender la diferencia entre los agentes `plan` y `build`.
- Configurar permisos básicos.

Tareas

- ☐ Leer Agentes y usar `plan` para una tarea de análisis.
- ☐ Crear un `opencode.json` de proyecto con reglas de permisos (ver Configuración).
- ☐ Revisar un cambio con `/undo` y `/redo` en un repositorio Git.

Comprobación: sabes qué agente usar para explorar y cuál para editar, y por qué.

Semana 3: Extensión

Objetivos

- Reutilizar instrucciones con skills y comandos.
- Conectar un servidor MCP.

Tareas

- ☐ Crear una skill de proyecto (ver Skills).
- ☐ Crear un comando personalizado (ver Comandos).
- ☐ Añadir un servidor MCP con `opencode mcp add` (ver MCP).

Comprobación: la skill se activa cuando la pides y el servidor MCP aparece en `opencode mcp list`.

Semana 4: Flujo con especificaciones y migración

Objetivos

- Planificar un cambio con especificaciones antes de implementarlo.
- Dejar el entorno de equipo listo.

Tareas

- ☐ Seguir OpenCode + OpenSpec en un cambio pequeño.
- ☐ Revisar Workflows.
- ☐ Si vienes de la versión 1, completar Migrar de V1 a V2.

Comprobación: un cambio pasa de propuesta a archivado sin pasos manuales ocultos.

Recursos

- Referencia de comandos
- Preguntas frecuentes
- Documentación oficial

Agentes

Agentes integrados de OpenCode V2 y cómo crear los tuyos

Agentes integrados

Agente	Modo	Función
`build`	`primary`	Agente por defecto para programar; puede editar archivos
`plan`	`primary`	Explora y planifica sin editar los ficheros normales del proyecto
`general`	`subagent`	Investigación y tareas de varios pasos; no puede lanzar más subagentes
`explore`	`subagent`	Busca y lee código o fuentes web sin editar archivos

Hay además agentes ocultos (`compaction`, `title` y `summary`) que gestionan tareas internas. No se pueden seleccionar.

Elegir el agente primario

Por defecto se usa `build`. Para cambiarlo:

```
{
  "$schema": "https://opencode.ai/config.json",
  "default_agent": "plan"
}
```

Al ejecutar desde la terminal, `opencode run --agent plan "..."` usa un agente concreto para esa petición.

Si el agente indicado no existe, no es visible o no puede usarse como primario, OpenCode recurre a `build`.

Subagentes

Un agente primario puede delegar en un subagente. Se pide en lenguaje natural:

```
Usa el subagente explore para localizar dónde se valida el correo
electrónico.
```

Los subagentes se ejecutan en sesiones hijas. Las reglas de permisos del agente padre controlan a cuáles puede llamar.

Agentes personalizados

Un agente es un fichero Markdown en `.opencode/agents/<nombre>.md` (proyecto) o `~/.config/opencode/agents/<nombre>.md` (global). El nombre del fichero es el identificador del agente.

```
---
description: Revisa cambios buscando errores de lógica y de seguridad
mode: subagent
permissions:
  - { "action": "edit", "resource": "*", "effect": "deny" }
steps: 20
---

Eres un revisor de código. Lee los cambios indicados y devuelve una lista
de problemas por severidad. No modifiques ningún fichero.
```

Campos admitidos en el frontmatter: `description`, `mode` (`primary`, `subagent` o `all`), `model` (formato `proveedor/modelo`), `permissions`, `steps`, `hidden`, `color` y `disabled`. El cuerpo del fichero es el prompt de sistema.

También puedes definir agentes en `opencode.json`, bajo la clave `agents`, con el campo `system` para el prompt:

```
{
  "$schema": "https://opencode.ai/config.json",
  "agents": {
    "revisor": {
      "description": "Revisa cambios sin editar",
      "mode": "subagent",
      "system": "Eres un revisor de código. No modifiques ficheros."
    }
  }
}
```

Evita los campos de la versión 1 (`temperature`, `top_p`, `prompt`, `permission`, `tools`, `maxSteps`). Usa `permissions` y `steps` en su lugar.

Permisos por agente

Cada agente puede tener sus propias reglas, en la clave `agents.<id>.permissions` o en el frontmatter:

```
{
  "$schema": "https://opencode.ai/config.json",
  "agents": {
    "plan": {
      "permissions": [
        { "action": "edit", "resource": "*", "effect": "deny" }
      ]
    }
  }
}
```

Consulta Configuración para la sintaxis completa de `permissions`.

Comprobar los agentes cargados

```
opencode debug agents
```

Lista todos los agentes que OpenCode ha cargado, incluidos los personalizados. Si tu agente no aparece, revisa el nombre del fichero, la ruta y el frontmatter.

Buenas prácticas

- Un agente, una responsabilidad. Un revisor no debería poder editar archivos.
- Limita `steps` en agentes de investigación para evitar sesiones muy largas.
- Usa `plan` para diseñar y `build` solo cuando el plan esté aprobado.
- En este repositorio, el `.gitignore` ignora `.opencode/agents/`. Si quieres compartir agentes con el equipo, ajusta esa regla.

CAPÍTULO 05 · GUÍAS

Configuración

Ficheros opencode.json, ámbitos, permisos y substituciones en OpenCode V2

OpenCode V2 se configura con un fichero JSON o JSONC. Los comentarios solo son válidos en `.jsonc`.

Dónde vive la configuración

Ámbito	Ruta
Global (todos los proyectos)	`~/.config/opencode/opencode.json` o `opencode.jsonc`
Proyecto	`opencode.json` o `opencode.jsonc` en la raíz, o `.opencode/opencode.json`

Para ver qué fuentes se están cargando y en qué orden:

```
opencode debug config
```

Estructura básica

```
{
  "$schema": "https://opencode.ai/config.json",
  // Modelo por defecto, en formato proveedor/modelo
  "model": "proveedor/modelo",
  "default_agent": "build",
  "update": "notify"
}
```

Claves principales de V2: `model`, `default_agent`, `shell`, `update`, `permissions`, `agents`, `commands`, `mcp`, `skills`, `instructions` y `providers`.

`update` admite `disable`, `notify` o `auto`.

Permisos

`permissions` es una lista ordenada de reglas con `action`, `resource` y `effect`. El efecto puede ser `allow`, `ask` o `deny`. Gana la última regla que coincide, y si ninguna coincide se pregunta.

```
{
  "$schema": "https://opencode.ai/config.json",
  "permissions": [
    { "action": "read", "resource": "*", "effect": "allow" },
    { "action": "edit", "resource": "*", "effect": "ask" },
    { "action": "edit", "resource": ".env*", "effect": "deny" },
    { "action": "shell", "resource": "git status*", "effect": "allow" },
    { "action": "shell", "resource": "*", "effect": "ask" }
  ]
}
```

Acciones habituales: `read`, `edit`, `shell`, `webfetch`, `websearch`, `glob`, `grep`, `skill`, `subagent`, `question`, `external_directory` y `execute`. Las herramientas de servidores MCP usan `<servidor>_<herramienta>`.

Los permisos también pueden definirse por agente, en `agents.<id>.permissions`. Consulta Agentes.

Sustituciones

Dentro de los valores puedes usar:

- `{env:NOMBRE}`: valor de una variable de entorno.
- `{file:ruta}`: contenido de un fichero.

Claves de V1 que ya no se usan

Las claves `permission`, `agent`, `command` y `provider` están obsoletas en V2. Sustitúyelas por `permissions`, `agents`, `commands` y `providers`. La guía Migrar de V1 a V2 muestra la correspondencia completa.

CAPÍTULO 06 · GUÍAS

Inicio rápido

Instala OpenCode V2, conecta un proveedor y ejecuta tu primera sesión

Requisitos

- Terminal con soporte para TUI (WezTerm, Alacritty, Windows Terminal o similar).
- Una cuenta de proveedor de modelos, o una cuenta en OpenCode Console (ver Zen / Console).

1. Instalar

Elige el método según tu sistema operativo.

macOS y Linux: script oficial

```
curl -fsSL https://opencode.ai/v2/install | bash
```

macOS y Linux: Homebrew

```
brew install anomalyco/tap/opencode-v2
```

Con un gestor de paquetes de Node.js

```
npm install -g @opencode/cli
```

Equivalentes con otros gestores: `bun install -g --trust @opencode/cli`, `pnpm add -g --allow-build=@opencode/cli @opencode/cli`, `yarn global add @opencode/cli`.

Arch Linux (AUR): `paru -S opencode-beta`

mise: `mise use -g opencode`

Windows

Los gestores de paquetes **no son compatibles** en Windows nativo. Tienes dos opciones:

- Descargar el binario independiente o el instalador de escritorio desde opencode.ai.
- Usar WSL y seguir las instrucciones de Linux dentro de la distribución.

2. Comprobar la instalación

```
opencode --version
```

3. Conectar un proveedor

Abre la TUI en tu proyecto y ejecuta:

```
cd /ruta/a/tu/proyecto
opencode
```

Dentro de la TUI, escribe `/connect` y sigue el asistente para añadir un proveedor. Para OpenCode Console, elige el proveedor **OpenCode** y pega tu clave de API.

4. Preparar el proyecto

Dentro de la TUI:

```
/init
```

Este comando genera un `AGENTS.md` con el contexto del proyecto. Revísalo y ajústalo a mano.

5. Primera petición

```
¿Qué hace este proyecto? @README.md
```

La referencia `@` busca ficheros del proyecto de forma aproximada.

También puedes lanzar una petición sin abrir la TUI:

```
opencode run "Resume la estructura de este proyecto"
```

Actualizar

```
opencode upgrade
```

Dentro de la TUI puedes usar `/update`. La clave `update` de la configuración controla si OpenCode avisa o instala actualizaciones automáticamente (`disable`, `notify` o `auto`).

Desinstalar

Revisa primero qué se eliminará:

```
opencode uninstall --dry-run
opencode uninstall
```

La desinstalación conserva las sesiones, la configuración y el estado.

Siguientes pasos

- TUI: atajos y comandos de barra.
- Configuración: `opencode.json` y permisos.
- Agentes: cómo usar `build` y `plan`.
- Si vienes de la versión 1, lee Migrar de V1 a V2.

CAPÍTULO 07 · GUÍAS

Skills

Instrucciones reutilizables en formato SKILL.md para OpenCode V2

Una skill es una carpeta con un fichero `SKILL.md`. Contiene instrucciones que el agente carga cuando las necesita, sin ocupar contexto en todas las sesiones.

Dónde se buscan

Proyecto

- `.opencode/skills/`
- `.claude/skills/`
- `.agents/skills/`

Global

- `~/.config/opencode/skills/`
- `~/.claude/skills/`
- `~/.agents/skills/`

Cada skill vive en su propio directorio:

```
.opencode/skills/revision-seguridad/
└── SKILL.md
```

Formato

```
---
name: revision-seguridad
description: Revisa cambios buscando problemas de seguridad. Úsala al
revisar código que maneje autenticación, datos de usuario o entrada externa.
---

# Revisión de seguridad

1. Busca credenciales o secretos escritos en el código.
2. Comprueba que las entradas externas se validan antes de usarse.
3. Revisa las consultas a base de datos por concatenación de cadenas.
4. Resume los hallazgos por severidad.
```

Campos del frontmatter: `name`, `description` (obligatoria para que la skill sea visible), `license`, `compatibility`, `metadata` y `disable-model-invocation`.

Uso

- **El modelo** la carga con la herramienta `skill` cuando la descripción encaja con la tarea.
- **Tú** la referencias por su identificador con `@revision-seguridad`.

Si `disable-model-invocation` está activo, solo podrás invocarla tú.

Configuración

Para cargar rutas adicionales, usa la clave `skills` de `opencode.json`. Consulta Configuración.

Buenas prácticas

- Escribe la `description` como un disparador: di cuándo debe usarse la skill.
- Mantén cada skill centrada en una tarea. Dos skills pequeñas se activan mejor que una grande.
- Versiona las skills de proyecto en Git, salvo que contengan datos sensibles.
- No guardes secretos ni claves de API en `SKILL.md`.

TUI

Atajos de teclado, comandos de barra y gestión de sesiones en la interfaz de terminal de OpenCode V2

Tecla líder

Muchos comandos tienen atajo con la tecla líder `ctrl+x`. Pulsa `ctrl+x` y, en un plazo de 2 segundos, la tecla de la acción.

Atajo	Comando	Función
`ctrl+x c`	`/compact`	Resume la conversación para liberar contexto
`ctrl+x e`	`/editor`	Abre un editor externo para escribir el mensaje
`ctrl+x m`	`/models`	Cambia de modelo
`ctrl+x n`	`/new`	Empieza una sesión nueva
`ctrl+x l`	`/sessions`	Lista y reanuda sesiones
`ctrl+x r`	`/redo`	Rehace el último cambio deshecho
`ctrl+x u`	`/undo`	Deshace el último cambio (requiere Git)
`ctrl+x t`	`/themes`	Cambia el tema
`ctrl+x x`	`/export`	Exporta la conversación
`ctrl+x q`	`/exit`	Sale de la TUI

Otros atajos:

- `ctrl+p`: paleta de comandos.
- `ctrl+t`: cambia entre las variantes del modelo actual.

Comandos de barra

Escribe `/` en la entrada para ver la lista. Estos son los comandos documentados en V2:

Comando	Alias	Función
`/connect`		Añade un proveedor
`/help`		Muestra la ayuda
`/init`		Genera `AGENTS.md` con el contexto del proyecto
`/thinking`		Muestra u oculta los bloques de razonamiento
`/update`		Actualiza OpenCode
`/summarize`	`/compact`	Resumen de la conversación
`/clear`	`/new`	Nueva sesión
`/quit`	`/q`, `/exit`	Salir
`/resume`	`/continue`	Reanudar sesiones

Los comandos personalizados también aparecen en esta lista. Consulta Comandos.

Referencias a ficheros

Escribe `@` seguido de parte del nombre para buscar un fichero del proyecto:

```
¿Qué hace la validación de formularios en @src/utils/validation.ts?
```

Sesiones

- Las sesiones se guardan y puedes reanudarlas con `/sessions` o `ctrl+x l`.
- Desde la terminal, `opencode session list` muestra las sesiones del proyecto y `opencode session delete` elimina una.

- `opencode session export` y `opencode session import` permiten copiar sesiones entre equipos.

Agente de la sesión

OpenCode V2 usa `build` como agente primario por defecto. Puedes fijar otro con la clave `default_agent` de la configuración, o indicarlo con `--agent` al ejecutar `opencode run`. Consulta Agentes.

Pensamiento del modelo

`/thinking` alterna la visualización de los bloques de razonamiento. Para que se muestren en modo no interactivo, usa `opencode run --thinking`.

Solución de problemas

- **La TUI no se ve bien.** Prueba otra terminal con soporte de color y Unicode.
- `ctrl+x` **no hace nada.** Comprueba que tu terminal no reserva ese atajo. El plazo de la tecla líder es de 2 segundos.
- `/undo` **no funciona.** Requiere que el proyecto sea un repositorio Git.

Zen y OpenCode Console

Cómo acceder a modelos de IA mediante OpenCode Console, con pago por uso

Zen se llama ahora **OpenCode Console**. Es un servicio de pago por uso que da acceso a modelos de IA con una sola clave de API.

Gestiona tu cuenta, tu saldo y tus claves en opencode.ai/console. Los precios y los modelos disponibles cambian con frecuencia, así que consulta siempre la consola.

Conectar la cuenta

1. Crea una cuenta en opencode.ai/console y genera una clave de API.
2. Abre OpenCode en tu proyecto y ejecuta `/connect`.
3. Elige el proveedor **OpenCode** y pega la clave.

Usar un modelo de Console

Los modelos de Console se referencian con el prefijo `opencode/`:

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "opencode/<id-del-modelo>"
}
```

Para ver los identificadores disponibles desde la terminal:

```
curl https://opencode.ai/inference/v1/models
```

Cambiar de modelo

- En la TUI: `ctrl+x m` (`/models`).
- Por proyecto o global: clave `model` de la configuración.
- Por petición: `opencode run --model proveedor/modelo "..."`.

Otros proveedores

Puedes conectar otros proveedores con `/connect`. La clave `providers` de la configuración permite ajustarlos. Consulta Configuración.

Comprobar la conexión

- Revisa el saldo y el uso en la consola.
- Si la clave no funciona, vuelve a ejecutar `/connect` y comprueba que pegaste la clave completa, sin espacios.
- `opencode auth list` muestra las integraciones y credenciales guardadas.

CAPÍTULO 10 · AVANZADO

Referencia de comandos

Comandos de la CLI de OpenCode V2, comandos de barra y comandos personalizados

CLI

```
opencode [directorio]              # Abre la TUI en el directorio
indicado
opencode run "mensaje"             # Ejecuta una petición sin abrir la TUI
opencode models                    # Lista los modelos disponibles
opencode auth list                 # Lista credenciales e integraciones
opencode session list              # Lista las sesiones del proyecto
opencode serve --port 4096         # Arranca el servidor de API y web
opencode stats                     # Estadísticas de uso
opencode mcp list                  # Estado de los servidores MCP
opencode debug config              # Fuentes de configuración cargadas
opencode debug agents              # Agentes cargados
opencode debug paths               # Rutas de datos, configuración y caché
opencode upgrade                   # Actualiza OpenCode
opencode uninstall --dry-run       # Muestra qué se eliminaría
```

`opencode run`

Flag	Función
`-m, --model proveedor/modelo#variante`	Modelo a usar
`--agent <nombre>`	Agente a usar
`--format default|json`	Formato de salida
`-f, --file <ruta>`	Adjunta un fichero al mensaje
`-c, --continue`	Continúa la última sesión
`-s, --session <id>`	Continúa o crea la sesión indicada
`--fork`	Bifurca la sesión antes de continuar
`-w, --worktree [nombre]`	Inicia la sesión en un worktree nuevo de Git
`--title <texto>`	Título de la sesión
`--thinking`	Muestra los bloques de razonamiento
`--auto`	Aprueba automáticamente los permisos no denegados explícitamente

Ejemplo para CI, en formato JSON:

```
opencode run --agent plan --format json "Lista los TODO del proyecto"
```

`opencode mcp`

```
opencode mcp add <nombre> -- <comando> [args...]
opencode mcp add <nombre> --url https://ejemplo.com/mcp
opencode mcp list
opencode mcp auth <nombre>
opencode mcp logout <nombre>
```

Consulta Servidores MCP.

Para una lista completa de subcomandos y flags, usa `opencode --help` o `opencode <subcomando> --help`.

Comandos de barra

Los comandos de la TUI están documentados en TUI. Escribe `/` para verlos todos, incluidos los personalizados.

Comandos personalizados

Un comando es una plantilla que se invoca como `/nombre` en la TUI.

Como fichero Markdown: `.opencode/commands/<nombre>.md` (proyecto) o `~/.config/opencode/commands/<nombre>.md` (global). Solo se admiten ficheros `.md`.

```
---
description: Revisa un fichero concreto
agent: plan
---

Revisa @$1 y lista los problemas de legibilidad. Contexto adicional:
$ARGUMENTS
```

Uso: `/revisar src/app.ts prioriza el manejo de errores`

Marcadores disponibles:

- `$ARGUMENTS`: todo lo que escribes tras el comando.
- `$1`, `$2`, …: argumentos posicionales.
- `!`comando``: ejecuta un comando de shell y sustituye su salida.

Como JSON, bajo la clave `commands`:

```
{
  "$schema": "https://opencode.ai/config.json",
  "commands": {
    "revisar": {
      "description": "Revisa un fichero concreto",
      "template": "Revisa @$1 y lista los problemas de legibilidad.",
      "agent": "plan"
    }
  }
}
```

Campos: `template` (obligatorio), `description`, `agent`, `model` y `subagent`. La clave `subtask` está obsoleta; usa `subagent`.

Si un comando de proyecto y uno global tienen el mismo nombre, gana el de proyecto.

Solución de problemas

- **El comando no aparece.** Comprueba la ruta y que el fichero termine en `.md`. Luego revisa `opencode debug config`.
- **Un marcador no se sustituye.** Usa `$ARGUMENTS` o `$1` exactamente así, en mayúsculas y con el signo `$`.

CAPÍTULO 11 · AVANZADO

FAQ y solución de problemas

Respuestas a preguntas frecuentes y diagnóstico de problemas en OpenCode V2

Instalación

¿Por qué no encuentro `opencode` tras instalar?

Comprueba dónde está el binario y que su directorio está en el `PATH`:

```
which opencode
opencode debug paths bin
```

Si instalaste V1 antes, puede haber dos binarios. Desinstala la versión que no uses.

¿Puedo usar gestores de paquetes en Windows?

No en V2. Descarga el binario o el instalador de escritorio desde opencode.ai, o usa WSL.

¿Puedo usar mi configuración de V1?

No directamente. Las claves cambiaron y `mcp.servers` no existe en V1. Sigue la guía Migrar de V1 a V2.

Configuración

¿Qué configuración se está cargando?

```
opencode debug config
```

Muestra las fuentes en el orden en que se aplican.

Aparece un aviso de clave obsoleta

Sustituye `permission`, `agent`, `command` o `provider` por `permissions`, `agents`, `commands` o `providers`. Consulta Configuración.

Proveedores

La clave de API no funciona

Vuelve a ejecutar `/connect` y pega la clave completa. Comprueba el saldo en OpenCode Console si usas esa cuenta.

```
opencode auth list
```

Uso

Las referencias `@archivo` no encuentran el fichero

Comprueba el directorio desde el que arrancaste OpenCode. Usa una parte del nombre o una ruta relativa al proyecto.

OpenCode pide permiso para todo

Revisa la lista `permissions` de tu configuración. Recuerda que, si ninguna regla coincide, se pregunta. Gana siempre la última regla que coincide.

`/undo` no funciona

El proyecto debe ser un repositorio Git.

Agentes y comandos

Mi agente no aparece

```
opencode debug agents
```

Revisa el nombre del fichero, su ruta (`.opencode/agents/` o `~/.config/opencode/agents/`) y el frontmatter.

Mi comando `/nombre` no aparece

Comprueba que el fichero es `.md` y está en `.opencode/commands/` o `~/.config/opencode/commands/`.

MCP

El servidor MCP no arranca

```
opencode mcp list
```

Ejecuta el `command` del servidor directamente en la terminal para ver el error real.

Actualizaciones

```
opencode upgrade
```

Desde la TUI puedes usar `/update`.

Ayuda

Para una referencia completa, consulta la documentación oficial. Si el problema persiste, incluye en tu consulta la versión de OpenCode (`opencode --version`) y la salida de `opencode debug config` con las claves sensibles eliminadas.

CAPÍTULO 12 · AVANZADO

Servidores MCP

Conecta OpenCode V2 con servidores MCP locales o remotos

Un servidor MCP (Model Context Protocol) expone herramientas externas al agente. OpenCode las carga al arrancar y las ofrece junto con sus propias herramientas.

Configuración

Los servidores van bajo `mcp.servers`. Cada uno es **local** (un proceso que arranca OpenCode) o **remoto** (una URL).

```jsonc
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "timeout": {
      "startup": 10000,
      "catalog": 10000,
      "execution": 60000
    },
    "servers": {
      "archivos": {
        "type": "local",
        "command": ["npx", "-y", "@ejemplo/servidor-mcp"],
        "environment": {
          "RUTA_BASE": "{env:HOME}/proyectos"
        },
        "disabled": false
      },
      "documentacion": {
        "type": "remote",
        "url": "https://ejemplo.com/mcp",
        "headers": {
          "Authorization": "Bearer {env:DOCS_TOKEN}"
        },
        "disabled": false
      }
    }
  }
}
```

Los valores de `timeout` están en milisegundos. `disabled: true` desactiva un servidor sin borrar su configuración.

Para servidores remotos con OAuth, la clave `oauth` admite `client_id`, `client_secret`, `scope`, `callback_port`, `redirect_uri` y `auth_server_metadata_url`.

Añadir desde la terminal

```
opencode mcp add archivos -- npx -y @ejemplo/servidor-mcp
opencode mcp add documentacion --url https://ejemplo.com/mcp --header
"Authorization=Bearer ..."
```

Flags de `opencode mcp add`:

- `--url`: URL de un servidor remoto.
- `--header clave=valor`: cabecera HTTP para un servidor remoto.
- `--env clave=valor`: variable de entorno para un servidor local.
- `--global`: escribe en la configuración global en lugar de la del proyecto.

No escribas secretos en el fichero de configuración. Usa `{env:NOMBRE}` y define la variable en tu entorno.

Comprobar y autenticar

```
opencode mcp list
opencode mcp auth documentacion
opencode mcp logout documentacion
```

`auth` solo aplica a servidores remotos que usan OAuth.

Nombres de las herramientas

Cada herramienta de un servidor aparece como `<servidor>_<herramienta>`. Por ejemplo, la herramienta `leer` del servidor `archivos` se llama `archivos_leer`.

Esos nombres se usan en las reglas de permisos:

```
{
  "$schema": "https://opencode.ai/config.json",
  "permissions": [
    { "action": "archivos_leer", "resource": "*", "effect": "allow" },
    { "action": "documentacion_*", "resource": "*", "effect": "ask" }
  ]
}
```

Migración desde V1

En V1, los servidores se declaraban como `mcp.<nombre>` con `enabled`. En V2 van bajo `mcp.servers.<nombre>` con `disabled`. Consulta Migrar de V1 a V2.

Solución de problemas

- **El servidor no arranca.** Ejecuta `opencode mcp list` para ver su estado. Prueba el comando de `command` directamente en la terminal.
- **Las herramientas no aparecen.** Comprueba que `disabled` es `false` y que el nombre del servidor no tiene caracteres extraños.
- **Error de autenticación.** Revisa la variable de entorno referenciada con `{env:...}` y vuelve a ejecutar `opencode mcp auth`.
- **Arranque lento.** Aumenta `mcp.timeout.startup`.

CAPÍTULO 13 · AVANZADO

Migrar de V1 a V2

Pasos y equivalencias para pasar de opencode-ai 1.x a @opencode/cli

OpenCode V2 se distribuye como `@opencode/cli`. La versión 1 se distribuye como `opencode-ai`. Los dos paquetes **no son intercambiables**, y la configuración de V1 no funciona tal cual en V2.

1. Instalar V2

Método V1	Equivalente V2
`npm install -g opencode-ai`	`npm install -g @opencode/cli`
`curl -fsSL https://opencode.ai/install | bash`	`curl -fsSL https://opencode.ai/v2/install | bash`
`brew install anomalyco/tap/opencode`	`brew install anomalyco/tap/opencode-v2`
`bun install -g opencode-ai`	`bun install -g --trust @opencode/cli`

En Windows, V2 no admite gestores de paquetes. Usa el binario o el instalador de escritorio, o WSL.

Desinstala V1 con el mismo gestor que usaste para instalarla, para evitar dos binarios `opencode` en el `PATH`.

2. Convertir la configuración

Pasa tu `opencode.json` de V1 a V2 con esta correspondencia:

V1	V2	Notas
`permission` (objeto `{ "herramienta": "allow" }`)	`permissions` (lista de reglas)	Cada regla tiene `action`, `resource` y `effect`
`mcp.<nombre>` con `enabled`	`mcp.servers.<nombre>` con `disabled`	`disabled` invierte la lógica de `enabled`
`agent`	`agents`	Clave de nivel superior
`command`	`commands`	Usa `template` en lugar de un texto suelto
`provider`	`providers`	
`subtask` en comandos	`subagent`	`subtask` queda como alias obsoleto

Dentro de los ficheros de agentes:

Campo de V1	Qué hacer en V2
`tools`	Sustituir por `permissions`
`temperature`	Eliminar
`prompt`	Mover el texto al cuerpo del fichero `.md` o al campo `system`
`maxSteps`	Sustituir por `steps`

Un ejemplo de MCP en V2:

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "servers": {
      "docs": {
        "type": "remote",
        "url": "https://ejemplo.com/mcp",
        "disabled": false
      }
    }
  }
}
```

Las claves `permission`, `agent`, `command` y `provider` siguen siendo aceptadas por compatibilidad, pero están obsoletas.

3. Validar

```
opencode debug config
```

Este comando muestra las fuentes de configuración que se cargan. Si una clave no se reconoce, aparecerá ahí antes de que falle una sesión.

4. Comprobar las sesiones y los comandos

- Lista las sesiones con `opencode session list`.
- Exporta una sesión importante antes de desinstalar V1 con `opencode session export`.
- Mueve tus comandos personalizados de `.opencode/commands/` a la nueva estructura si usas la forma JSON.

Qué no migra automáticamente

- La configuración global de V1 no se lee desde V2. Copia y convierte el fichero manualmente.
- Las reglas de permisos hay que reescribirlas como lista ordenada. Recuerda que gana la última regla que coincide.

Lista de comprobación

- ☐ V2 instalado y `opencode --version` responde.
- ☐ V1 desinstalado o fuera del `PATH`.
- ☐ `opencode.json` convertido y sin claves obsoletas.
- ☐ `opencode debug config` muestra las fuentes esperadas.
- ☐ Servidores MCP arrancan (`opencode mcp list`).
- ☐ Una sesión de prueba completa sin errores de permisos.

CAPÍTULO 14 · AVANZADO

Flujos de trabajo

Un flujo recomendado para planificar, implementar y revisar cambios con OpenCode

---

Un flujo sencillo reduce los errores de las herramientas de programación con IA: primero se entiende el código, después se planifica y, por último, se implementa.

Flujo recomendado

```
Explorar (plan)  →  Planificar (plan)  →  Implementar (build)  →
Revisar  →  Confirmar
```

1. Explorar con `plan`

El agente `plan` lee el código sin editarlo.

```
Explica cómo se gestiona la sesión de usuario en @src/auth/ y qué
módulos dependen de ella.
```

2. Planificar

Pide un plan concreto y revisa las preguntas que haga el agente:

```
Propón un plan para añadir expiración a las sesiones. Lista los ficheros
que cambiarían y los riesgos.
```

Si el cambio es grande, usa OpenCode + OpenSpec para dejar el plan por escrito antes de tocar código.

3. Implementar con `build`

Cuando el plan esté aprobado, cambia de agente con `default_agent` o con `opencode run --agent build`. Implementa una fase cada vez:

```
Implementa solo la primera tarea del plan. Detente al terminar para que pueda revisarla.
```

4. Revisar

Pide la revisión a un subagente o a un comando personalizado, con permisos de solo lectura:

```
Usa el subagente explore para comprobar que los cambios no rompen las llamadas a validateSession.
```

5. Confirmar

Los cambios se confirman con Git como cualquier otro código. Antes de hacer commit, revisa el diff con tus herramientas habituales.

Deshacer cambios

- `ctrl+x u` (`/undo`) deshace el último cambio del agente, y `ctrl+x r` (`/redo`) lo rehace.
- Requiere que el proyecto sea un repositorio Git. Sin Git, usa tu control de versiones normal.

Buenas prácticas

- Limita los permisos de `edit` y `shell` en los agentes que no necesitan escribir.

- Usa `/compact` cuando la conversación sea larga, en lugar de empezar una sesión con todo el historial.
- Crea una sesión nueva (`/new`) para cada tarea independiente.
- Guarda las convenciones del proyecto en `AGENTS.md` para no repetirlas en cada sesión.

CAPÍTULO 15 · OPENCODE + OPENSPEC

OpenCode + OpenSpec

Cómo combinar OpenCode con OpenSpec para planificar cambios antes de programarlos

---

OpenSpec es un flujo de trabajo de desarrollo guiado por especificaciones. Antes de tocar código, el agente escribe una propuesta, las especificaciones del cambio, un diseño y una lista de tareas. Después implementa esas tareas y, al final, fusiona las especificaciones en la documentación principal del proyecto.

Esta sección explica cómo usarlo con OpenCode. Para el detalle completo del método, consulta la **Guía completa de OpenSpec** ↗️.

Qué aporta a OpenCode

- **Planificación separada de la implementación.** Los artefactos de planificación se generan primero y se revisan antes de escribir código.
- **Especificaciones del proyecto.** Los cambios se acumulan en `openspec/specs/` y sirven de contexto para cambios futuros.
- **Comandos reutilizables.** `openspec init --tools opencode` instala comandos `/opsx-*` y skills en el proyecto.

Flujo general

Paso	Comando en OpenCode	Qué produce
Explorar (opcional)	`/opsx-explore`	Ideas y preguntas aclaradas, sin artefactos
Proponer	`/opsx-propose "idea"`	`proposal.md`, specs delta, `design.md` y `tasks.md`
Implementar	`/opsx-apply`	Código que marca las tareas como completadas
Archivar	`/opsx-archive`	Fusiona las specs delta y mueve el cambio a `changes/archive/`

Los comandos completos y su descripción están en Flujo de trabajo.

Requisitos

- OpenCode instalado (ver Inicio rápido).
- Node.js para instalar el CLI de OpenSpec por npm, o el binario standalone.
- Un proyecto con Git. Los cambios se revisan como cualquier otro código.

Siguientes pasos

1. Instalación y configuración en tu proyecto.
2. Flujo de trabajo con explorar, proponer, implementar y archivar.
3. Comandos del CLI para consultar y validar cambios desde la terminal.

Comandos del CLI

Comandos de la terminal de OpenSpec para listar, validar y archivar cambios

Estos comandos se ejecutan en la terminal, desde la raíz del proyecto. Son independientes de OpenCode, aunque se usan juntos.

Comando	Para qué sirve
`openspec list`	Lista los cambios activos
`openspec list --specs`	Lista las especificaciones principales
`openspec view`	Abre un panel interactivo con specs y cambios
`openspec show <elemento>`	Muestra un cambio o una spec
`openspec status`	Muestra qué artefactos de un cambio están completos
`openspec validate [elemento]`	Valida cambios y specs
`openspec archive <cambio>`	Archiva un cambio y actualiza las specs principales
`openspec update`	Regenera los archivos de instrucciones tras actualizar el CLI
`openspec config profile`	Activa workflows adicionales
`openspec init --tools opencode`	Configura el proyecto para OpenCode

Revisa siempre las opciones con `openspec <comando> --help`, porque el CLI evoluciona con frecuencia.

Ejemplo de revisión antes de un PR

```
openspec list
openspec validate add-passwordless-login
openspec status --change add-passwordless-login
```

Si `validate` falla, corrige los artefactos con `/opsx-update` en OpenCode antes de implementar o archivar.

Flujo de trabajo

Explorar, proponer, implementar y archivar un cambio con los comandos /opsx-* de OpenCode

Comandos disponibles

Comando	Para qué sirve
`/opsx-explore`	Pensar una idea o investigar un problema sin crear artefactos
`/opsx-propose "idea"`	Crear el cambio y generar todos sus artefactos de una vez
`/opsx-apply [cambio]`	Implementar las tareas de `tasks.md` y marcarlas como hechas
`/opsx-archive [cambio]`	Archivar un cambio terminado
`/opsx-sync [cambio]`	Fusionar las specs delta en las specs principales sin archivar
`/opsx-update [cambio]`	Revisar los artefactos de un cambio y mantenerlos coherentes

`apply`, `archive` y `update` aparecen marcados como *experimentales* en su descripción. Revisa la salida de cada paso.

Paso a paso

1. Explorar (opcional)

Úsalo cuando la idea no está clara:

```
/opsx-explore ¿Cómo debería funcionar el inicio de sesión sin
contraseña?
```

Ejecútalo con el agente `plan`, que no edita archivos del proyecto.

2. Proponer

```
/opsx-propose "añadir inicio de sesión sin contraseña"
```

El comando crea la carpeta del cambio en `openspec/changes/` con:

- `proposal.md`: qué se quiere y por qué.
- `specs/<ruta>/spec.md`: especificaciones delta (qué cambia en el comportamiento del sistema).
- `design.md`: cómo se va a implementar.
- `tasks.md`: lista ordenada de tareas.

Revisa los artefactos antes de implementar. Es el momento más barato para corregir un error de planteamiento.

3. Implementar

```
/opsx-apply add-passwordless-login
```

Ejecútalo con el agente `build`, que sí puede editar archivos. El agente recorre `tasks.md` y marca cada tarea al completarla.

4. Archivar

```
/opsx-archive add-passwordless-login
```

El archivado fusiona las specs delta en `openspec/specs/` y mueve el cambio a `openspec/changes/archive/`.

Práctica recomendada en equipo: archivar dentro del mismo PR antes de fusionarlo, para que la documentación principal quede alineada con el código.

Agentes recomendados

Fase	Agente	Motivo
Explorar y revisar propuestas	`plan`	Solo lectura
Implementar	`build`	Necesita editar archivos

Puedes fijar el agente por defecto con `default_agent` en `opencode.json`, o indicarlo al ejecutar en modo no interactivo con `opencode run --agent plan "..."`. Consulta Agentes.

Flujo sin la TUI

Los comandos también pueden lanzarse con `opencode run`:

```
opencode run --agent build "/opsx-apply add-passwordless-login"
```

Errores habituales

- **El comando `/opsx-*` no aparece.** Comprueba que `.opencode/commands/` no esté ignorado por Git (ver Instalación) y que se ejecuta desde la raíz del proyecto.
- **Se usa `/opsx:propose`.** En OpenCode el separador es un guion.
- **Las specs principales no cambian tras implementar.** Falta ejecutar `/opsx-archive` o `/opsx-sync`.

Instalación y configuración

Instala OpenSpec y prepara un proyecto para usarlo desde OpenCode

1. Instalar el CLI de OpenSpec

Con npm:

```
npm install -g @fission-ai/openspec@latest
```

También puedes instalar el binario standalone, que no necesita Node.js en tiempo de ejecución:

```
openspec install
```

Comprueba la instalación con `openspec version`.

2. Inicializar el proyecto para OpenCode

Desde la raíz del proyecto:

```
openspec init --tools opencode
```

Para que los artefactos se escriban en otro idioma, usa `--language`:

```
openspec init --tools opencode --language es
```

Revisa las opciones disponibles con `openspec init --help`.

3. Qué se crea

Ruta	Contenido
`.opencode/commands/opsx-*.md`	Comandos: `opsx-propose`, `opsx-apply`, `opsx-archive`, `opsx-explore`, `opsx-sync`, `opsx-update`
`.opencode/skills/openspec-*/SKILL.md`	Skills que implementan esos flujos
`openspec/config.yaml`	Configuración del proyecto (esquema `spec-driven`)
`openspec/specs/`	Especificaciones principales del proyecto
`openspec/changes/archive/`	Cambios ya archivados

En OpenCode, los comandos usan **guion** (`/opsx-propose`). En Claude Code se escriben con dos puntos (`/opsx:propose`). No mezcles ambas formas en la documentación del equipo.

4. Configurar el proyecto

Edita `openspec/config.yaml` para añadir restricciones que el agente no pueda deducir del código. Por ejemplo:

```yaml
schema: spec-driven

context: |
  Los diseños y las tareas deben cubrir Windows, macOS y Linux.
  Escribe todos los artefactos en español.

rules:
  tasks:
    - Divide las tareas en bloques de como máximo 2 horas
```

Describe solo restricciones de negocio o de entorno. La documentación general del código no hace falta, porque el agente la puede leer.

5. Workflows adicionales

Por defecto se instalan los flujos principales. Los workflows `new`, `continue`, `ff`, `bulk-archive`, `verify` y `onboard` se añaden con:

```
openspec config profile
```

6. Actualizar tras una nueva versión

Cuando actualices el CLI, regenera los archivos de instrucciones:

```
openspec update
```

Aviso: el `.gitignore` puede ocultar los comandos al equipo

Si el `.gitignore` del proyecto ignora `.opencode/commands/`, `.opencode/skills/` o `.opencode/agents/`, los comandos `/opsx-*` no se compartirán con el resto del equipo. En este repositorio, el `.gitignore` raíz ignora esas tres carpetas, así que conviene revisarlo antes de hacer commit.

Para compartir el flujo, elimina esas reglas del `.gitignore` del proyecto y añade al control de versiones las carpetas `.opencode/commands/` y `.opencode/skills/`, junto con `openspec/`.

Puedes comprobar qué regla aplica con:

```
git check-ignore -v .opencode/commands/opsx-propose.md
```

Recursos

Documentación oficial y guías de esta web para aprender OpenCode

Documentación oficial

- **Web oficial**: opencode.ai
- **Documentación**: opencode.ai/docs
- **OpenCode Console** (cuentas y claves de API): opencode.ai/console

Guías de esta web

Guía	Descripción
Inicio rápido	Instalación y primera sesión
TUI	Atajos, comandos de barra y sesiones
Configuración	`opencode.json` y permisos
Agentes	Agentes integrados y personalizados
Skills	Instrucciones reutilizables
Zen / Console	Acceso a modelos
Comandos	CLI y comandos personalizados
MCP	Servidores de herramientas externos
Workflows	Flujos de trabajo recomendados
FAQ	Problemas comunes
Migrar de V1 a V2	Pasos para actualizar desde `opencode-ai`
OpenCode + OpenSpec	Especificaciones antes de programar

Si encuentras un error en esta guía, revisa primero la documentación oficial, que es la fuente de referencia.

Diseño y desarrollo web: webdespega.com